RNAiMachinerySearch: rapid and optimized screening of core genes from RNA interference machinery

User's Guide

Gustavo Fernando Ferreira Gonçalves[1], Isabela dos Santos Begnami[2]

[1] Federal University of São Carlos, São Carlos, São Paulo, Brazil

[2] State University of Campinas, Campinas, São Paulo, Brazil

First edition 15 December 2025

Contents

Glossary

Annotation: The process of providing biological meaning to DNA or RNA sequences, such as identifying genes, predicting coding regions, and associating functional information (e.g., protein names, GO terms).

Category (in RNAi context): A functional class of genes involved in the RNA interference mechanism, such as Dicer-like enzymes, Argonautes, RNA-dependent RNA polymerases, and silencing cofactors.

Contig: A contiguous sequence of RNA that has been assembled from overlapping sequencing reads, representing a reconstructed portion of the transcriptome.

CPM: Counts Per Million, a normalized measure of RNA-seq expression that scales raw read counts by the total number of reads in the sample, expressed per million reads, to enable comparison across samples.

CPM threshold: A cutoff value used to filter out genes with low expression, ensuring that only biologically relevant transcripts are retained in downstream analyses.

dsRNA: double stranded RNA.

HQ reads: high-quality reads; sequencing reads retained after quality control filters based on metrics such as base-calling accuracy, Phred quality scores, read length, etc.

logCPM: A normalization method that applies a logarithmic transformation to CPM values, stabilizing variance and improving visualization of expression data.

Normalization: A mathematical adjustment of expression data to reduce technical variation between samples, allowing fair comparison of gene expression levels. Examples include logCPM and Z-score transformations.

Read: A short sequence of nucleotides generated by a sequencing platform, representing a fragment of the original RNA molecule.

RNAi: RNA interference, a biological process in which small RNA molecules inhibit gene expression by causing degrading specific messenger RNA (mRNA) molecules or blocking their translation.

RNAseq: RNA sequencing, a high-throughput sequencing technique used to profile the complete set of RNA transcripts (the transcriptome) in a sample, allowing quantification of gene expression and identification of novel transcripts.

Z-score: A standardization method that expresses how many standard deviations a value is from the mean of its dataset; useful for comparing expression patterns across genes or samples.

1. Introduction

This guide provides an overview of the RNAiMachinerySearch package, a rapid and optimized tool for screening the machinery of genes responsible for executing RNA interference (RNAi) in eukaryotes. The package provides a set of functions to investigate transcriptomic datasets annotated with UniProtKB/Swiss-Prot coming from RNA-seq experiments and retrieve information such as: how many genes from each category of function in RNAi are present; what is the profile of expression of these genes across experimental groups; and which category of genes (understanded as functions and steps of response) is being most requested in each experimental group. It currently targets five core RNAi machinery categories: Dicer-like proteins, Argonaute/Piwi proteins, double-stranded RNA-binding proteins, double-stranded RNA-transport/uptake proteins, and auxiliary RISC-associated proteins.

The package automates the process of identifying and classifying RNAi core components (e.g., Dicer, Argonaute, Drosha, Piwi) based on UniProt identifiers curated from current and relevant references on the topic. Using CPM thresholds and group-level replicates, the filtering process ensures that only representative contigs are retained in outputs, making the visualization charts ready-to-use in publications or presentations.

2. Installation

The RNAiMachinerySearch package can be installed directly from GitHub using the devtools package in R:

```r
# Install devtools if not already installed
install.packages("devtools")

# Load devtools package
library(devtools)

# Install RNAiMachinerySearch from GitHub
devtools::install_github("GustavoGoncalvesF/RNAiMachinerySearch")

# Load the package
library(RNAiMachinerySearch)
```

To operate properly, RNAiMachinerySearch depends on the following R packages (which will be automatically installed when you get it from GitHub): `dplyr`, `plotly`, `stringr`, `BiocManager`, `ComplexHeatmap`, `edgeR` and `openxlsx`.

3. Input

Considering a standard workflow of RNA-seq projects, a large amount of reads will first be filtered to keep only those considered suitable for use, then these high-quality reads can be used in two distinct approaches to assemble a transcriptome: *de novo* assembly (without any reference, such as a previously performed genome) or reference-based assembly. In both scenarios, at some point, the data set (organized in the form of contigs) will be annotated in terms of protein function through the functional annotation process and mapped by the initial reads to obtain sample counts, thereby quantifying expression. These comprise the main inputs for RNAiMachinerySearch.

3.1. Functional annotation report

The search strategy implemented by this package is based on UniProt names from RNAi proteins as identifiers in your data; therefore, the first input must be a dataset annotated with the UniProt database, usually created by programs such as Trinotate, EnTAP, eggNOG-mapper, etc. In general, this software produces a file determined as an "annotation report", which contains information such as gene/transcripts ID, best hits via BLASTx/BLASTp (or protein names mapped by ontology), etc. (Figure 1).

Figure 1: Example of an annotation report file.

gene_id	sprot_Top_BLASTX_hit	Pfam	prot_coords	...
TRINITY_DN1000_c0_g1_i1	sp|P60709|ACTB_HUMAN Actin, cytoplasmic 1	PF00022:Actin	1-376	...
TRINITY_DN1054_c1_g2_i2	sp|Q9Y6K9|HSP90A_HUMAN Heat shock protein	PF00183:HSP90	5-732	...
TRINITY_DN1103_c0_g1_i1	sp|P08047|TBA1_MOUSE Tubulin alpha-1 chain	PF00091:Tubulin	15-449	...
TRINITY_DN1205_c2_g1_i3	sp|P49674|EF1A_DROME Elongation factor 1	PF03143:EF1_alpha	3-458	...
TRINITY_DN1302_c0_g2_i2	sp|Q8W3K0|CATZ_HUMAN Cathepsin Z	PF00112:Peptidase_C1	27-305	...

These annotation files are usually provided in tabular formats, most commonly as Excel spreadsheets (.xls, .xlsx) or plain text tables (.csv, .tsv). These files contain structured information organized in columns and rows, making them suitable for direct

import into R as data frames — the fundamental data structure used for handling tabular data.

Depending on the file format, distinct functions can be used to load the data. For plain text files, such as .csv and .tsv, the base R functions `read.csv()` or `read.table()` are typically employed. For Excel spreadsheets, such as .xls or .xlsx, the `read_excel()` function from the `readxl` package is recommended. Regardless of the format, the goal is to obtain a clean and properly structured data frame that has exactly the same form and content as the original file (Figure 1) and can be used directly in RNAi gene search functions.

3.2. Expression matrix

With a transcriptome assembly, we obtain information about the presence and structure of contigs derived from all HQ reads. After functional annotation, these contigs can be associated with specific genes or proteins. However, to understand how their expression changes across samples or conditions, expression quantification is required.

Modern quantification tools do this estimation of transcripts abundance in each sample through alignment-based or alignment-free approaches: alignment-based methods (e.g., HTSeq, featureCounts) count reads mapped to genomic features, while alignment-free tools (such as Salmon, Kallisto, and RSEM) estimate transcript abundances using lightweight algorithms based on k-mer matching or quasi-mapping.

This process will result in a table file (usually .txt, .csv or .tsv) containing the raw counts of each transcript in each sample. Here we call it an "expression matrix" (Figure 2).

Figure 2: Example of an expression matrix file.

	GroupA1	GroupA2	GroupA3	GroupB1	GroupB2	GroupB3	GroupC1	GroupC2	GroupC3
TRINITY_DN1000_c0_g1_i1	312	298	325	1120	1087	1155	890	865	912
TRINITY_DN1054_c1_g2_i2	1578	1602	1555	980	1008	962	1850	1904	1822
TRINITY_DN1103_c0_g1_i1	45	51	47	20	19	18	30	32	28
TRINITY_DN1205_c2_g1_i3	2200	2255	2189	3105	3150	3087	3800	3725	3892
TRINITY_DN1302_c0_g2_i2	5	7	6	60	55	62	45	47	43
...	...	...	...	...	...	...	...	...	...

The input of this file in R is done similarly to the previous one, using functions like `read.csv()` or `read.table()`. This results in a second data frame with the same form and content as Figure 2, which we will use to perform the filtering and plotting steps.

3.3. Sample grouping

The third file we will need is a description of the grouping between the samples, highlighting the different groups and their respective replicates. This can be done with a text file (.txt), written according to the following models (Figure 3):

Figure 3: Three different models to use for sample grouping file.

Model 1	Model 2	Model 3
SAMPLE,REP	SAMPLE,REP	SAMPLE, REP
GroupA,1	GroupControl,1	GroupAControl,1
GroupA,2	GroupControl,2	GroupAControl,2
GroupA,3	GroupControl,3	GroupAControl,3
GroupB,1	Condition1,1	GroupBControl,1
GroupB,2	Condition1,2	GroupBControl,2
GroupB,3	Condition1,3	GroupBControl,3
GroupC,1	Condition2,1	GroupACondition1,1
GroupC,2	Condition2,2	GroupACondition1,2
GroupC,3	Condition2,3	GroupACondition1,3
GroupD,1	Condition3,1	GroupBCondition1,1
GroupD,2	Condition3,2	GroupBCondition1,2
GroupD,3	Condition3,3	GroupBCondition1,3
		GroupACondition2,1
		GroupACondition2,2
		GroupACondition2,3
		GroupBCondition2,1
		GroupBCondition2,2
		GroupBCondition2,3

Because of the specific way that the package reads metadata, the sample grouping input must follow the examples as shown in models of Figure 3. Only by using this structure can the functions correctly access and process the data.

- In Model 1, the experiment includes four groups (A, B, C, and D), each one with three replicates;
- In Model 2, there is one control group and three treatment conditions (1, 2, and 3), each one with three replicates;
- In Model 3, the experiment contains two groups (A and B) tested under two conditions (1 and 2), plus a control condition, each one with three replicates;

The essential requirement is that the sample grouping file (usually named as `samples.txt`) must include a `SAMPLE` column whose entries exactly match the prefix of the sample names in the expression matrix header, and the replicate identifiers (`REP`) must also be consistent with this.

4. Key functions

The package provides a streamlined workflow to identify and analyze RNAi-related genes in transcriptomic datasets. After preparing the input files, users can run a series of functions that will automatically search, organize, filter, and summarize candidate genes associated with RNA interference pathways.

4.1. `search.rnai()`

This function takes an annotation data frame and the name of the column containing UniProt identifiers. Using a curated catalog of RNAi-related genes grouped by functional categories, it searches for matches and returns the core components of the RNAi machinery present in the dataset.

Usage

```
# Function that search RNAi genes in annotation data frame
raw_rnai_hits <- search.rnai(annotation_df,"column")
```

Arguments

- `annotation_df` – the first input, a data frame containing gene IDs, functional annotations, etc.;

- `column` – it is a string that refers exactly to the column name in `annotation_df` where the UniProt names are located;

Output

- He prints a "`Summary of identified RNAi core machinery genes`" that show how many genes are founded in total and in each of five categories;
- Returns a data frame compound by three columns: `GeneID`, `ProteinAnnotation`, and `Category`;

Details

- This function scans the data frame column for matches against predefined RNAi-related terms and gene identifiers, so the output can be considered as "raw RNAi hits" because no expression filters are applied until now;
- Different contigs with the same UniProt name annotation will be kept here with a prefix ".1, .2, […]", so in return all contigs relating to machinery will be represented;

4.2. expr.filter()

This function takes a raw RNAi hits data frame, an expression data frame, a sample grouping data frame, and two threshold parameters to perform a CPM-based filtering, retaining only contigs with relevant expression levels.

Usage

```
# Function that filter the raw RNAi hits data frame to keep only relevant contigs
in downstream steps
filtered_rnai_hits <- expr.filter(raw_rnai_hits, expression_df, groups_df,
cpm_cut_group = 1, cpm_cut_global = 10)
```

Arguments

- `raw_rnai_hits` – a data frame coming from `search.rnai()`;
- `expression_df` – the second input, a data frame containing samples names in the header, one contig per row and raw counts in the cells;
- `groups_df` – the third input, a data frame containing sample grouping of the experiment;
- `cpm_cut_group` *(optional)* – group minimum expression criterion, define a number ≥ 1;
- `cpm_cut_global` *(optional)* – global minimum expression criterion, define a number ≥ 10;

Output

- He prints a "`Report of genes filtering`" that shows how many genes we had before filtering, how many were discarded by the group filter, how many were discarded by the global filter, and how many remain after filtering;
- Returns a data frame with the same format as `raw_rnai_hits`, but only with the contigs remaining after filtering;

Details

- The group filter is set to keep only contigs with a CPM ≥ `cpm_cut_group` in all samples of almost one group;
- The global filter is set to keep only contigs with a CPM ≥ `cpm_cut_global` in at least n samples, where n is the minimum number of replicates in the experiment;
- Arguments `cpm_cut_group` and `cpm_cut_global` are optional, and their default parameters are 1 and 10, respectively;

4.3. sunburst.plot()

To initiate our data visualization, this function generates a sunburst plot of the RNAi hits. The sunburst chart provides an interactive catalog of the RNAi machinery genes detected in your transcriptome, allowing users to explore each category, component, and cellular function in a hierarchical structure.

Usage

```
# Function that plots a sunburst chart
sunburst.plot(rnai_hits, save = FALSE, path = NULL)
```

Arguments

- `rnai_hits` – The only input here is a data frame of RNAi hits, usually `raw_rnai_hits` or `filtered_rnai_hits`;
- `save` *(optional)* – Define as `TRUE` if you want to save the plot output in a .html file;
- `path` *(optional)* – Define the path to save the .html file (if you do not, will save in repository path;

Output

- It generates a sunburst map with found genes, categorized into five functions within the machinery, with their respective assigned cellular functions. The chart can be visualized directly in the R plotting window and, if `save = TRUE`, is also exported as an .html file.

Details

- `save` and `path` are optional, and they default parameters are `FALSE` and `NULL`, respectively;
- You can plot two sunburst's to compare before and after filtering by `expr.filter()`;

4.4. stackedbars.plot()

To extend the data visualization, this function generates a stacked bar plot illustrating the distribution of RNAi-related genes across experimental groups. This chart highlights how each RNAi machinery category contributes within and between sample groups, enabling a clear comparison of functional representation and abundance patterns in the dataset.

Usage

```
# Function that plots a stacked bars chart
stackedbars.plot(rnai_hits, expression_df, groups_df, save_table = TRUE,
table_path = NULL, save_plot = TRUE, plot_path = NULL)
```

Arguments

- `rnai_hits` - A data frame of RNAi hits, usually `raw_rnai_hits` or `filtered_rnai_hits`;
- `expression_df` - a data frame containing samples names in the header, one contig per row and raw counts in the cells (same as `expr.filter()`);
- `groups_df` - a data frame containing sample grouping of the experiment (same as `expr.filter()`);
- `save_table` *(optional)* - Define as `TRUE` if you want to save a table (.xlsx) with the total counts by category;
- `save_plot` *(optional)* - Define as `TRUE` if you want to save the plot output in a .html file;
- `table_path` *(optional)* – Define the path to save the .xlsx file (if you don't, will save in repository path;
- `plot_path` *(optional)* – Define the path to save the .html file (if you don't, will save in repository path;

Output

- Generates a stacked bar chart showing the distribution of counts from RNAi machinery categories across sample groups. The chart can be visualized directly in the R plotting window and, if `save_plot = TRUE`, is also exported as an .html file.
- If `save_table = TRUE`, it also exports a table containing the total counts per RNAi category for each group.

Details

- `save_table`, `table_path`, `save_plot`, and `plot_path` are optional, and they default parameters are `FALSE`, `NULL`, `FALSE` and `NULL`, respectively;

4.5. heatmap.plot()

This function generates a heatmap to visualize the expression profiles of core RNAi genes across all samples. It extracts only the RNAi hits present in expression matrix, normalizes their expression values, and displays them in a structured matrix format, allowing users to easily compare expression patterns, identify co-expressed genes, and detect sample-specific differences in RNAi machinery activity.

Usage

```
# Function that plots a heatmap
heatmap.plot(rnai_hits, expression_df, normalize = "zscore", save = TRUE, path = NULL)
```

Arguments

- `rnai_hits` - A data frame of RNAi hits, usually `raw_rnai_hits` or `filtered_rnai_hits`;
- `expression_df` - a data frame containing samples names in the header, one contig per row and raw counts in the cells (same as `expr.filter()`);
- `normalize` – Defines the normalization method applied before plotting. We have three options: "`logCPM`", converts raw count to log2-CPM; "`zscore`", scales expression values across genes; and "`none`", uses raw counts without transformation.
- `save` *(optional)* – Define as `TRUE` if you want to save the plot output in a .png file;
- `path` *(optional)* – Define the path to save the .png file (if you do not, will save in repository path;

Output

- Returns a heatmap object representing the expression of RNAi core genes across samples. The chart can be visualized directly in the R plotting window and, if `save = TRUE`, is also exported as an image file.

Details

- `save` and `path` are optional, and their default parameters are `FALSE` and `NULL`, respectively;
- `normalize` defaults to "`logCPM`", which is the most commonly used normalization method for RNA-seq count data in this context.

5. General workflow

This is a didactic example of a simple workflow of the package:

5.1. Import and organize inputs

```
# Input annotation table
file1 <- file.path("path/to/annotation_report.xls")
annotation_df <- read.table(file1, sep = "\t", header = TRUE, quote = "",
comment.char = "", fill = TRUE)

# Input expression matrix
file2 <- file.path("path/to/raw_counts.txt")
expression_df <- read.table(file2, header = TRUE, row.names = 1, sep = " ",
stringsAsFactors = FALSE)

# Input groups file
file3 <- file.path("path/to/samples.txt")
groups_df <- read.table(file3, header = TRUE, sep = ",", stringsAsFactors = FALSE)
```

This step generates three data frames in R: the RNAi hits annotation report, the expression matrix, and the sample grouping table.

5.2. Search for RNAi core genes and filter by expression

```
# Searching RNAi core genes
raw_rnai_hits <- search.rnai(annotation_df, "sprot_Top_BLASTX_hit")

# Filtering to retains biologically meaningful contigs
filtered_rnai_hits <- expr.filter(raw_rnai_hits, expression_df, groups_df,
cpm_cut_group = 1, cpm_cut_global = 20)
```

Now, two new data frames are generated: `raw_rnai_hits` which contains all the genes identified in the annotation dataset, and `filtered_rnai_hits`, which includes only the most relevant ones, suitable for subsequent analysis.

5.3. Visualize results

```
# Generate and export a sunburst chart to visualize filtered RNAi results
sunburst.plot(filtered_rnai_hits, save = TRUE, path =
"path/to/save/sunburst.html")

# Generate and export a stacked bars chart to visualize the distribution of RNAi-
related genes across experimental groups
stackedbars.plot(filtered_rnai_hits, expression_df, groups_df, save_plot = TRUE,
plot_path = "path/to/save/stackedbars.html")

# Generate and export two heatmaps to visualize the expression profiles of core
RNAi genes across all samples, before and after filtering
heatmap.plot(raw_rnai_hits, expression_df, normalize = "zscore")
heatmap.plot(filtered_rnai_hits, expression_df, normalize = "zscore", save = TRUE,
path = "path/to/save/filtered_heatmap.png")
```

Here we present our plots: a sunburst chart (Figure 4), a stacked bars chart (Figure 5) and two heatmaps (Figure 6 and Figure 7).

Figure 4: Sunburst chart in figure format.

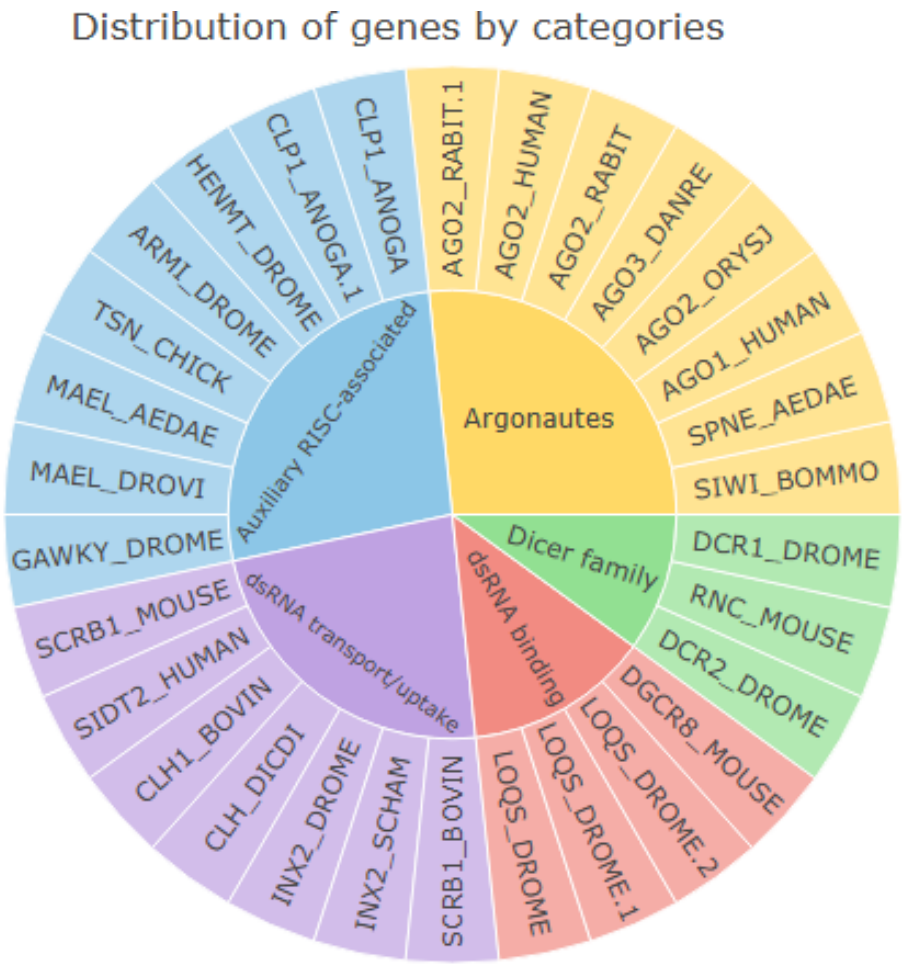


Figure 5: Stacked bars chart in figure format.

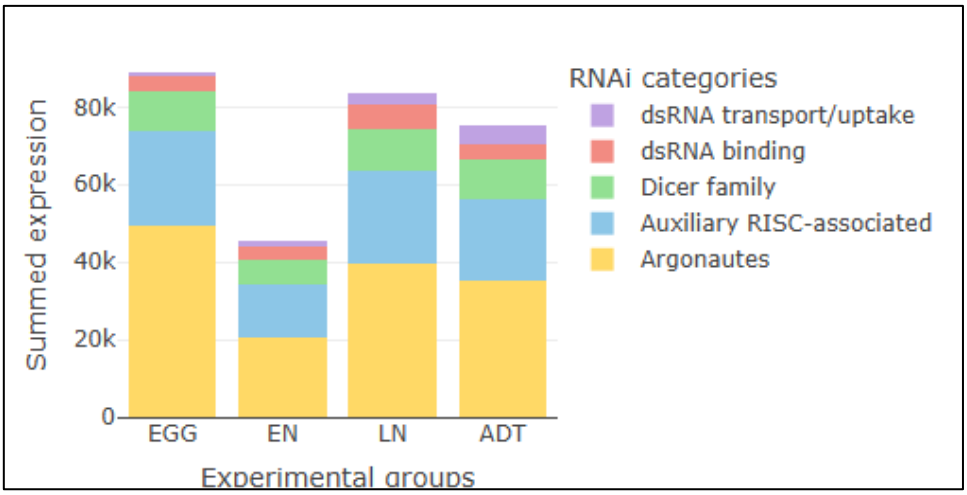


Figure 6: Heatmap with normalization by z-score and raw RNAi hits.

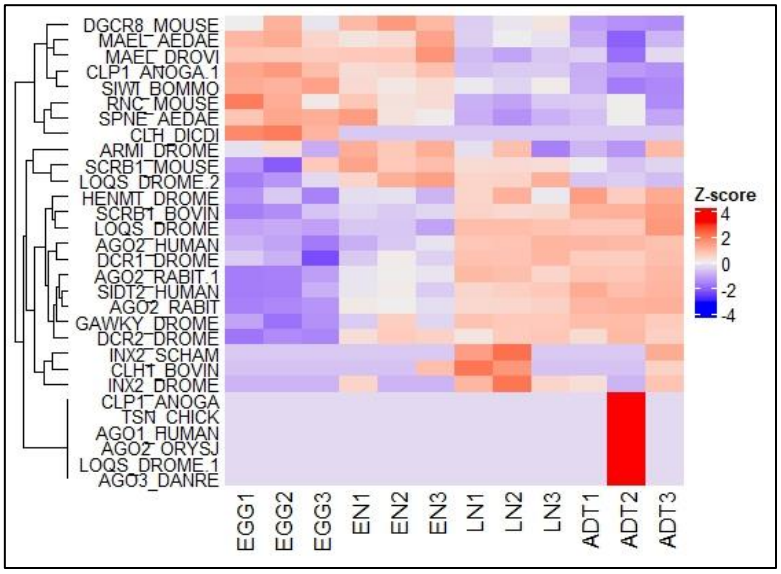
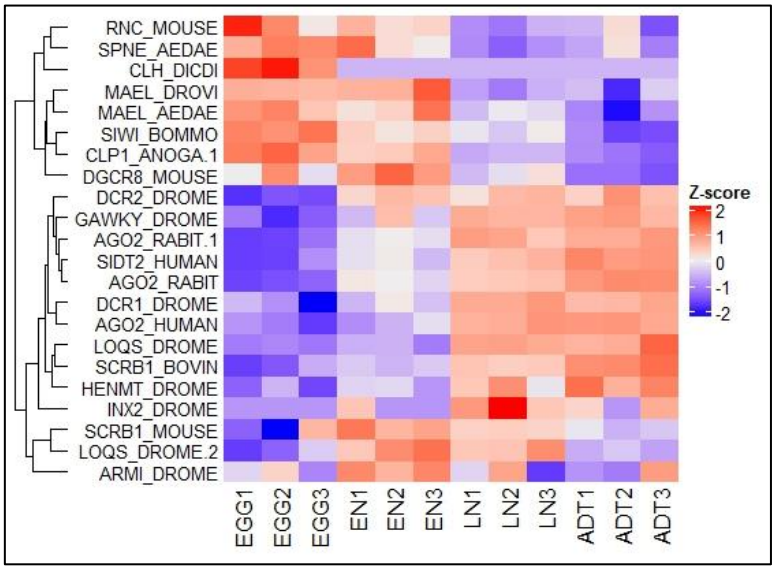


Figure 7: Heatmap normalization by z-score and filtered RNAi hits.



6. Troubleshooting

Below are some common issues users may encounter when running RNAiMachinerySearch, along with suggested solutions:

6.1. Input structure and data type errors

These occur when the main input objects are missing, malformed, or incorrectly formatted.

- **Error:** `RNAiMachinerySearch Error: 'annotation_df' must be a data frame.`
  - ✓ Ensure that your annotation report is loaded as a data frame (e.g., `read.csv()` or `read.table()`).

- **Error:** `RNAiMachinerySearch Error: 'expression_df' cannot be empty.`
  - ✓ Check if your expression matrix has both rows (genes/contigs) and columns (samples). Remove empty data frames.

- **Error:** `RNAiMachinerySearch Error: 'groups_df' must contain the columns: SAMPLE, REP.`
  - ✓ The grouping file must have exactly these two columns to describe experimental groups and their replicates.

- **Error:** `RNAiMachinerySearch Error: 'rnai_hits' must contain the columns: GeneID, ProteinAnnotation, Category.`
  - ✓ The RNAi hits table must retain these columns for downstream plots and summaries.

6.2. Column and content issues

- **Error:** `RNAiMachinerySearch Error: Column '<name>' not found in 'annotation_df'.`
  - ✓ Verify that the column name passed to the function matches exactly (case-sensitive) the column in your annotation report.

- **Error:** `RNAiMachinerySearch Error: Column '<name>' contains only missing values.`
  - ✓ Check if the chosen column in your annotation file has valid UniProt or annotation entries; otherwise, try another column.

6.3. Thresholds and normalization

- **Error:** `RNAiMachinerySearch Error: 'cpm_cut' must contain only non-negative numeric values.`

  ✓ The CPM cutoff must be numeric and ≥ 0. Avoid NA or character values.

- **Error:** `RNAiMachinerySearch Error: 'normalize' argument is required. Please choose one of: logCPM, zscore, none.`

  ✓ When using functions that support normalization (e.g., `heatmap.plot()`), specify the normalization method explicitly.

6.4. Saving output

- **Error:** `RNAiMachinerySearch Error: 'save' must contain only TRUE or FALSE values.`

  ✓ Make sure `save = TRUE` or `save = FALSE` (not character strings like `"TRUE"`).

- **Error:** `RNAiMachinerySearch Error: The following directories do not exist: ...`

  ✓ Create the specified directory before running the function.

7. Bibliography

Chen, Yunshun, et al. **"edgeR v4: powerful differential analysis of sequencing data with expanded functionality and improved support for small counts and larger datasets."** Nucleic Acids Research 53.2 (2025): gkaf018.

Gentleman, Robert C., et al. **"Bioconductor: open software development for computational biology and bioinformatics."** Genome Biology 5.10 (2004): R80.

Gu, Zuguang. **"Complex heatmap visualization."** Imeta 1.3 (2022): e43.

Jain, Ritesh G., et al. **"RNAi-based functional genomics in Hemiptera."** Insects 11.9 (2020): 557.

Kim, Daniel H., and John J. Rossi. **"RNAi mechanisms and applications."** Biotechniques 44.5 (2008): 613–616.

Mamta, B., and M. V. Rajam. **"RNAi technology: a new platform for crop pest control."** Physiology and Molecular Biology of Plants 23.3 (2017): 487–501.

R Core Team. **"R: A language and environment for statistical computing. R Foundation for Statistical Computing, Vienna, Austria."** http://www.R-project.org/ (2016).

Schauberger, P., and Walker, A. **"openxlsx: Read, Write and Edit xlsx Files."** R package version 4.2.8, Website: https://CRAN.R-project.org/package=openxlsx (2025).

Sievert, Carson, et al. **"Package 'plotly'."** R Foundation for Statistical Computing, Vienna (2021).

Wickham, Hadley, and Maintainer Hadley Wickham. **"Package 'stringr'."** Website: http://stringr.tidyverse.org, https://github.com/tidyverse/stringr (2019).

Wickham, Hadley, et al. **"Package 'devtools'."** R Package, Version 2.4 (2022).

Yarberry, William. **"Dplyr."** CRAN Recipes: DPLYR, stringr, lubridate, and regex in R. Berkeley, CA: Apress, 2021. 1–58.